Java OOP Laboratory Exercise: Campus Café Ordering System

ITS 202: Objected Oriented Programming

Submitted by:

Denzemiel Ethan S. Dacillo

Denemiel Nathan S. Dacillo

Section: C

NOVEMBER 9, 2026

Concept Explanation

1. **if Statement**: We used if statements in Pastry.computePrice() and Meal.computePrice() to add extra charges only when needed, such as when a pastry is warmed or a meal is upsized. We also used them in Order to check whether an item slot is empty before processing it.

2. **if-else Statement**: We used if-else statements when the program needed to choose between two outcomes. For example, we used them to reject negative prices in setBasePrice(), check the size of a beverage, limit the number of items in an order, and make sure the total does not become negative when applying a voucher.

3. **Classes and Objects**: Our program has seven classes, and each one is placed in its own Java file. We created the customer, menu item, and order objects in CafeApp, where we also demonstrated how the different parts of the ordering system work together.

4. **Attributes and Behaviors**: We used attributes to store information about each object, such as the size of a beverage or whether a pastry is warmed. We also created methods that define what an object can do, like calculating its price using computePrice() or adding items through addItem().

5. **Object Interaction (Order):** The Order class connects a customer with up to three menu items. When we call printReceipt(), it gets the details and prices of the ordered items, calculates the total, and updates the customer's loyalty points.

6. **Java Naming Conventions:** We followed PascalCase when naming our classes, such as MenuItem, and camelCase for methods and variables, such as basePrice and computeTotal(). For the additional fees, we directly wrote the amounts in the calculations instead of creating separate constants.

7. **Default Constructor**: We used default constructors to give objects their starting values when no information is provided. For example, MenuItem() sets the name to "Unnamed Item" and the base price to 0.0.

8. **Parameterized Constructor**: We used parameterized constructors to provide an object's information when creating it. In our subclasses, we also used super(...) to pass the item's name, price, and code to the parent MenuItem class.

9. **Constructor Overloading**: In the Customer class, we created three constructors so we could create a customer in different ways: without any information, with only a name, or with both a name and student status.

10. **public / private / protected**: We used different access modifiers depending on how each variable or method should be accessed. Most of our instance variables are private, while name and basePrice are protected so the subclasses can use them.

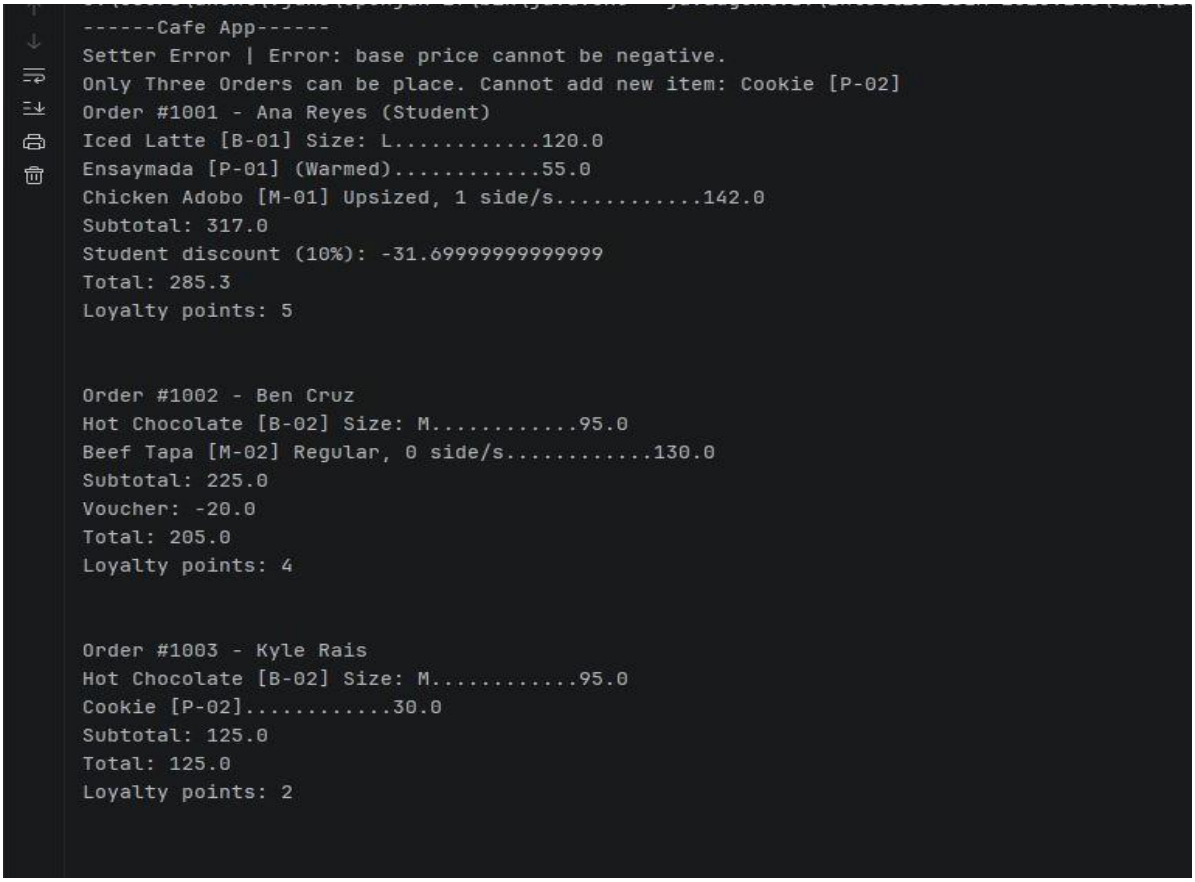
11. **Getters, Setters, and Data Hiding**: We used getters and setters to access and change the values of our attributes instead of making the variables public. We also added validation to some setters to prevent invalid values, such as negative prices and loyalty points.

12. **Method Overloading**: We applied method overloading by creating methods with the same name but different parameters. Some examples in our program are addPoints(int) and addPoints(double), computeTotal() and computeTotal(double), and printReceipt() and printReceipt(double).

13. **Method Overriding**: We used method overriding in our Beverage, Pastry, and Meal classes. Each class has its own implementation of computePrice() and getDescription() instead of simply using the versions inherited from MenuItem. We also added the @Override annotation to these methods.

14. **Abstraction:** We made MenuItem an abstract class because it serves as the parent class of our different menu items. Its abstract method computePrice() requires each subclass to provide its own way of calculating the price.

Screenshot / Output:



Challenges Encountered:

- **Comparing Strings**: In Beverage, we initially used == to compare the drink sizes. We also had trouble checking whether the input was "S", "M", or "L" because we combined the three values into one string. We fixed this by using .equals() for each comparison and connecting the conditions with ||.

- **Size Left Empty**: When we passed an invalid size to the Beverage constructor, the setter rejected it, leaving the size as null. This could cause an error when computePrice() was called. To fix this, we assigned "S" as the default size before checking the given input.

- **Abstract Method**: At first, we forgot to declare MenuItem as an abstract class and gave computePrice() a method body. We corrected this by adding the abstract keyword to the class and removing the method body, leaving only its declaration.

- **Points in Customer**: We initially made a mistake in our addPoints() methods because we were replacing the existing points instead of adding to them. Our negative-value checks were also incorrect, which caused valid values to be rejected. We fixed these by checking the incoming values first, using += to add the points, and casting the amount spent into whole points based on the rule of one point for every 50 pesos.

- **Using += and *= Correctly**: We also had some trouble with the operators when calculating the subtotal and total in the Order class. At first, we tried adding a value directly to the result of a method call, which Java does not allow. We fixed this by using variables to keep track of the subtotal and total, updating them during the calculations, and returning the correct values at the end of the methods.